

DRAFT — สำหรับตรวจสอบแนวทาง

# คู่มือการใช้งาน Mount TCP/IP JSON API

ข้อกำหนดเบื้องต้นสำหรับควบคุมแกน Azimuth/Altitude และรับข้อมูล  
Telemetry ผ่านเครือข่าย TCP/IP

เอกสาร: API Specification & User Guide

เวอร์ชัน: 0.1 Draft

วันที่: 22 กรกฎาคม 2026

พอร์ตที่เสนอ: TCP 7624

# สารบัญ

1. สถานะของเอกสารและขอบเขต
2. ภาพรวมการเชื่อมต่อ
3. โครงสร้างข้อความ JSON
4. ลำดับการใช้งานมาตรฐาน
5. รายการคำสั่ง
6. ระบบ Subscribe และข้อมูลที่ Mount ส่งออก
7. Error codes และความผิดปกติ
8. ตัวอย่าง Client ด้วย Python
9. Checklist สำหรับทดสอบ

**สำคัญ:** เอกสารนี้เป็นข้อเสนอ v0.1 สำหรับพิจารณารูปแบบการใช้งาน TCP API เท่านั้น ตัว TCP server และคำสั่งในเอกสารยังไม่ได้ถูกเปิดใช้งานบน Mount จนกว่าจะอนุมัติ specification และนำไปพัฒนา

## 1. ขอบเขตและหลักการออกแบบ

API นี้ออกแบบให้โปรแกรมภายนอกควบคุม Mount แบบสองแกน ได้แก่ **Azimuth** และ **Altitude** รวมถึงรับข้อมูลสถานะอย่างต่อเนื่อง โดยเน้นหลักดังต่อไปนี้

- ใช้ TCP/IP เพื่อรับประกันลำดับของข้อความและตรวจพบการตัดการเชื่อมต่อ
- ใช้ JSON เพื่ออ่านง่ายและรองรับหลายภาษาโปรแกรม
- คำสั่งทุกชุดมี id เพื่อจับคู่ Response
- ข้อมูล Telemetry ใช้ระบบ Subscribe แยกตามหัวข้อ
- คำสั่งเคลื่อนที่ต้องผ่าน motion limits และ safety guard ของ Mount
- มีผู้ครอบครองสิทธิ์ควบคุมการเคลื่อนที่ได้ครั้งละหนึ่ง Client

## 2. ภาพรวมการเชื่อมต่อ

| รายการ                | ค่าที่เสนอ                                                    |
|-----------------------|---------------------------------------------------------------|
| Transport             | TCP/IP                                                        |
| Default port          | 7624                                                          |
| Message framing       | NDJSON — JSON หนึ่ง Object ต่อหนึ่งบรรทัด ปิดท้ายด้วย LF (\n) |
| Character encoding    | UTF-8                                                         |
| API version           | 1.0                                                           |
| Recommended heartbeat | ทุก 1 วินาที                                                  |
| Maximum message size  | 64 KiB ต่อข้อความ                                             |

## 2.1 เหตุผลที่ใช้ NDJSON

TCP เป็น byte stream และไม่มีขอบเขตข้อความในตัวเอง การกำหนดให้ JSON ทุกชุดปิดท้ายด้วย newline ทำให้ Client อ่านที่ละบรรทัดและแยกข้อความได้ง่าย ห้ามส่ง JSON ที่จัดรูปแบบหลายบรรทัดผ่านสาย TCP

```
{
  "version": "1.0",
  "type": "command",
  "id": "cmd-1",
  "action": "system.hello",
  "params": {}
}
```

## 3. โครงสร้างข้อความ

### 3.1 Command จาก Client

```
{
  "version": "1.0",
  "type": "command",
  "id": "cmd-000123",
  "action": "mount.goto",
  "timestamp": "2026-07-22T13:30:00.000+07:00",
  "params": {
    "azimuth_deg": -120.5,
    "altitude_deg": 35.25,
    "max_velocity_deg_per_sec": 10
  }
}
```

| Field     | Required | ความหมาย                              |
|-----------|----------|---------------------------------------|
| version   | ใช่      | เวอร์ชัน Protocol                     |
| type      | ใช่      | ต้องเป็น command                      |
| id        | ใช่      | รหัสที่ไม่ซ้ำภายใน Connection         |
| action    | ใช่      | ชื่อคำสั่ง                            |
| timestamp | แนะนำ    | ISO 8601 พร้อม timezone               |
| params    | ใช่      | Object ของ Parameter; ใช้ {} หากไม่มี |

### 3.2 Response จาก Mount

```
{
  "version": "1.0",
  "type": "response",
  "id": "cmd-000123",
  "action": "mount.goto",
  "ok": true,
  "timestamp": "2026-07-22T13:30:00.020+07:00",
  "result": {"accepted": true}
}
```

ok: true หมายถึง Mount รับคำสั่งแล้ว ไม่ได้หมายความว่าเคลื่อนที่ถึงเป้าหมายแล้ว ให้ติดตามผลผ่านหัวข้อ motion.state

### 3.3 Error Response

```
{
  "version": "1.0",
  "type": "response",
  "id": "cmd-000123",
  "action": "mount.goto",
  "ok": false,
  "error": {
    "code": "TARGET_OUT_OF_RANGE",
    "message": "Altitude target exceeds the configured upper limit.",
    "details": {"requested_deg": 110, "maximum_deg": 100}
  }
}
```

### 3.4 Event จาก Mount

```
{
  "version": "1.0",
  "type": "event",
  "topic": "mount.status",
  "sequence": 1024,
  "timestamp": "2026-07-22T13:30:01.100+07:00",
  "data": {"connected": true, "enabled": false, "moving": false}
}
```

## 4. ลำดับการใช้งานมาตรฐาน

1. เปิด TCP connection ไปยัง IP ของ Mount ที่ port 7624
  2. ส่ง `system.hello` เพื่อตรวจ API version และ capabilities
  3. ส่ง `subscribe` เพื่อรับสถานะที่ต้องการ
  4. ส่ง `control.acquire` ก่อนคำสั่งที่ทำให้ Mount เคลื่อนที่
  5. ตรวจว่า Drive พร้อมและไม่มี Fault ผ่าน `mount.status`
  6. ส่ง `mount.enable`
  7. ส่งคำสั่ง `mount.goto` หรือ `axis.set_velocity`
  8. ติดตาม `motion.state` จนถึง `target_reached`
  9. เมื่อเลิกใช้งานให้ส่ง `mount.stop`, `mount.disable` และ `control.release`
- ข้อแนะนำ:** โปรแกรมควร Subscribe หัวข้อ `fault` เสมอ และต้องเตรียมส่ง `mount.stop` ได้ทันทีโดยไม่รอคำสั่งก่อนหน้า

## 5. รายการคำสั่ง

### 5.1 System และ Connection

| Action                  | หน้าที่                                       | สิทธิ์ควบคุม |
|-------------------------|-----------------------------------------------|--------------|
| system.hello            | อ่าน protocol version และ capabilities        | ไม่ต้องมี    |
| system.ping             | ตรวจ Connection/latency                       | ไม่ต้องมี    |
| system.get_info         | อ่านรุ่น, software version, device ID, uptime | ไม่ต้องมี    |
| system.get_health       | อ่านสุขภาพ Drive และ Fault ปัจจุบัน           | ไม่ต้องมี    |
| system.get_capabilities | อ่านคำสั่งและ Topic ที่รองรับ                 | ไม่ต้องมี    |

### 5.2 Control lease

| Action          | Parameter สำคัญ       | หน้าที่              |
|-----------------|-----------------------|----------------------|
| control.acquire | client_name, lease_ms | ขอสิทธิ์ควบคุม Mount |
| control.renew   | lease_id              | ต่ออายุสิทธิ์        |
| control.release | lease_id              | คืนสิทธิ์            |

```
{
  "version": "1.0", "type": "command", "id": "lease-1",
  "action": "control.acquire",
  "params": {"client_name": "Telescope Controller", "lease_ms": 5000}
}
```

## 5.3 เปิด ปิด และหยุด

| Action        | Parameter | รายละเอียด                                      |
|---------------|-----------|-------------------------------------------------|
| mount.enable  | —         | Enable ทั้งสองแกนที่พร้อมใช้งาน                 |
| mount.disable | —         | Disable ทั้งสองแกน                              |
| mount.stop    | —         | ยกเลิก Goto, Velocity, Tracking และ Calibration |
| axis.enable   | axis      | Enable แกนเดียว                                 |
| axis.disable  | axis      | Disable แกนเดียว                                |
| axis.stop     | axis      | หยุดแกนเดียว                                    |

## 5.4 Goto Position

```
{
  "version": "1.0", "type": "command", "id": "goto-1",
  "action": "mount.goto",
  "params": {
    "azimuth_deg": -120.5,
    "altitude_deg": 35.25,
    "max_velocity_deg_per_sec": 10
  }
}
```

สามารถส่งเป้าหมายเพียงแกนเดียวผ่าน axis.goto:

```
{
  "version": "1.0", "type": "command", "id": "goto-2",
  "action": "axis.goto",
  "params": { "axis": "altitude", "position_deg": 45, "max_velocity_deg_per_sec": 5 }
}
```

## 5.5 Velocity และ Jog

```
{
  "version": "1.0", "type": "command", "id": "vel-1",
  "action": "axis.set_velocity",
  "params": {
    "axis": "azimuth",
    "velocity_deg_per_sec": -5,
    "command_timeout_ms": 1000
  }
}
```

**Safety watchdog:** คำสั่ง Velocity/Jog ต้องมี timeout หาก Client ไม่ต่ออายุคำสั่งภายในเวลาที่กำหนด Mount ต้องสั่งความเร็วเป็นศูนย์โดยอัตโนมัติ

## 5.6 Motor Calibration

```
{
  "version": "1.0", "type": "command", "id": "cal-1",
  "action": "calibration.start",
  "params": {
    "axis": "altitude",
    "minimum_position_deg": 0,
    "maximum_position_deg": 45,
    "current_amp": 10,
    "velocity_deg_per_sec": 5
  }
}
```

หยุดด้วย `calibration.stop` หรือ `mount.stop` ค่า current limit ที่ใช้ทดสอบต้องไม่เกินขีดจำกัดที่ตั้งไว้ใน Mount

## 6. Subscribe และ Telemetry

### 6.1 เริ่ม Subscribe

```
{
  "version": "1.0", "type": "command", "id": "sub-1",
  "action": "subscribe",
  "params": {
    "topics": ["mount.status", "axis.telemetry", "motion.state", "sensor.status", "fault"],
    "interval_ms": 100
  }
}
```

Mount อาจปรับ interval\_ms ให้เข้ากับช่วงที่รองรับ และต้องส่งค่าที่ใช้จริงกลับใน Response การยกเลิกใช้ unsubscribe พร้อม subscription\_id

| Topic              | ข้อมูล                                                  | อัตราที่แนะนำ                    |
|--------------------|---------------------------------------------------------|----------------------------------|
| mount.status       | สถานะรวม, enable, moving, tracking, health, bus voltage | 2–5 Hz และเมื่อเปลี่ยน           |
| axis.telemetry     | ตำแหน่ง, ความเร็ว, กระแส, target, state, error แยกแกน   | 1–40 Hz ตามที่ขอ                 |
| motion.state       | โหมดเคลื่อนที่, target, error, progress                 | เมื่อเปลี่ยน + ระหว่างเคลื่อนที่ |
| sensor.status      | CCW/CW limit sensors                                    | เมื่อเปลี่ยน                     |
| drive.status       | serial, firmware, voltage, errors, disarm reason        | 1 Hz และเมื่อเปลี่ยน             |
| calibration.status | phase, target, position, current, peak current          | 5–10 Hz ขณะทำงาน                 |
| fault              | เหตุผิดปกติและการตอบสนองด้านความปลอดภัย                 | ส่งทันที                         |

### 6.2 ตัวอย่าง Axis Telemetry

```
{
  "version": "1.0", "type": "event", "topic": "axis.telemetry",
  "sequence": 1025, "timestamp": "2026-07-22T13:30:01.120+07:00",
  "data": {
    "axes": {
      "azimuth": {
        "available": true, "enabled": false,
        "position_deg": -155.85, "velocity_deg_per_sec": 0.002,
        "current_amp": 0, "target_position_deg": null,
        "state": 1, "errors": 0
      },
      "altitude": {
        "available": true, "enabled": false,
        "position_deg": 0.064, "velocity_deg_per_sec": 0,
        "current_amp": 0, "target_position_deg": null,
        "state": 1, "errors": 0
      }
    }
  }
}
```



### 6.3 ตัวอย่าง Motion State

```
{
  "version": "1.0", "type": "event", "topic": "motion.state",
  "sequence": 1026, "timestamp": "2026-07-22T13:30:02.000+07:00",
  "data": {
    "state": "moving", "command_id": "goto-1", "mode": "goto",
    "progress": { "azimuth_error_deg": 12.4, "altitude_error_deg": 3.1 }
  }
}
```

ค่าที่เสนอสำหรับ state: idle, moving, target\_reached, tracking, stopping, stopped, calibrating, fault

### 6.4 ตัวอย่าง Fault

```
{
  "version": "1.0", "type": "event", "topic": "fault",
  "sequence": 1027, "timestamp": "2026-07-22T13:30:03.000+07:00",
  "data": {
    "severity": "critical",
    "code": "AZIMUTH_CW_LIMIT_ACTIVE",
    "axis": "azimuth",
    "message": "Azimuth CW limit is active.",
    "motion_stopped": true,
    "requires_acknowledgement": true
  }
}
```

## 7. Error Codes

| Code                    | ความหมาย                 | แนวทาง Client                                 |
|-------------------------|--------------------------|-----------------------------------------------|
| INVALID_JSON            | JSON ไม่ถูกต้อง          | ตรวจ syntax และ newline framing               |
| UNSUPPORTED_VERSION     | API version ไม่รองรับ    | เรียก system.hello ใหม่ด้วย version ที่รองรับ |
| UNKNOWN_ACTION          | ไม่พบคำสั่ง              | ตรวจ capabilities                             |
| INVALID_PARAMETER       | Parameter ขาดหรือชนิดผิด | แก้ params ตาม details                        |
| CONTROL_LEASE_REQUIRED  | ยังไม่ได้สิทธิ์ควบคุม    | เรียก control.acquire                         |
| CONTROL_LEASE_BUSY      | Client อื่นควบคุมอยู่    | รอหรือทำงานแบบ read-only                      |
| DRIVE_NOT_CONNECTED     | ไม่พบ Drive ของแกน       | หยุดคำสั่งและแจ้งผู้ดูแล                      |
| AXIS_NOT_ENABLED        | แกนยังไม่ Enable         | ตรวจ health แล้วเรียก axis.enable             |
| TARGET_OUT_OF_RANGE     | ตำแหน่งเกิน limit        | ใช้ค่าขอบเขตจาก error details                 |
| VELOCITY_LIMIT_EXCEEDED | ความเร็วสูงเกินกำหนด     | ลดความเร็ว                                    |
| CURRENT_LIMIT_EXCEEDED  | ค่ากระแสสูงเกินกำหนด     | ลดกระแส                                       |
| MOUNT_FAULT             | Mount อยู่ใน Fault       | ส่ง stop และตรวจ fault topic                  |
| COMMAND_TIMEOUT         | คำสั่งหมดเวลา            | ตรวจ Connection และสถานะ Mount                |

## 8. Safety และ Security

---

- `mount.stop` ต้องเรียกได้เสมอ แม้ไม่มี control lease หรือ Mount อยู่ใน Fault
- TCP หลุดต้องยกเลิก Velocity/Jog/Calibration ที่ Connection นั้นเป็นผู้เริ่ม
- Goto ทุกครั้งต้องตรวจสอบ software limits และ limit sensors
- Mount ต้องจำกัดความเร็วและกระแสวิกชัน ไม่เชื่อคำจาก Client โดยตรง
- Control lease ต้องหมดอายุเองหาก Client ไม่ Renew
- หากใช้นอก LAN ควรใช้ TLS และ API token หรือ VPN
- จำกัด message size, connection count และ command rate
- บันทึก audit log ของคำสั่งเคลื่อนที่พร้อม Client identity

## 9. ตัวอย่าง Python Client

```
import json
import socket

HOST = "192.168.1.50"
PORT = 7624

def send_message(stream, message):
    wire = json.dumps(message, separators=(",", ":")) + "\n"
    stream.write(wire.encode("utf-8"))
    stream.flush()

with socket.create_connection((HOST, PORT), timeout=5) as sock:
    stream = sock.makefile("rwb")

    send_message(stream, {
        "version": "1.0",
        "type": "command",
        "id": "hello-1",
        "action": "system.hello",
        "params": {}
    })

    send_message(stream, {
        "version": "1.0",
        "type": "command",
        "id": "sub-1",
        "action": "subscribe",
        "params": {
            "topics": ["mount.status", "axis.telemetry", "fault"],
            "interval_ms": 200
        }
    })

    while True:
        line = stream.readline()
        if not line:
            raise ConnectionError("Mount disconnected")
        message = json.loads(line)
        print(message)
```

### 9.1 ตัวอย่าง Goto

```
send_message(stream, {
    "version": "1.0",
    "type": "command",
    "id": "goto-1",
    "action": "mount.goto",
    "params": {
        "azimuth_deg": -120.5,
        "altitude_deg": 35.25,
        "max_velocity_deg_per_sec": 5
    }
})
```

## 10. Checklist ก่อนใช้งานจริง

1. ยืนยัน serial mapping ของ Azimuth และ Altitude
2. ยืนยัน position offset และทิศทางบวก/ลบของแต่ละแกน

3. ทดสอบ CCW/CW sensors และ software limits
4. ทดสอบ mount.stop ระหว่าง Goto, Velocity และ Calibration
5. ทดสอบถอดสาย LAN ระหว่าง Velocity command
6. ทดสอบ control lease หมดอายุ
7. ทดสอบ Client สองตัวขอสิทธิ์พร้อมกัน
8. ทดสอบ JSON ไม่ถูกต้องและข้อความขนาดเกินกำหนด
9. ทดสอบ Subscribe หลายอัตราและ sequence gap
10. ยืนยันว่า Fault ถูกส่งทันทีและ Mount หยุดตามเงื่อนไข

## 11. ประเด็นที่ต้องตัดสินใจก่อนพัฒนา

---

- ต้องรองรับ RA/Dec และ Tracking ใน API รุ่นแรกหรือไม่
- ต้องการ Authentication แบบ token, TLS หรือจำกัดเฉพาะ LAN
- อัตรา Telemetry สูงสุดที่ต้องรองรับ
- พฤติกรรมเมื่อ Client ผู้ควบคุมหลุด: Stop, Hold หรือ Disable
- อนุญาตให้แก้ Tuning/Settings ผ่าน TCP หรือให้เป็น read-only
- ต้องการรองรับ compatibility กับ ASCOM/INDI ในอนาคตหรือไม่

จบเอกสาร — Mount TCP/IP JSON API, Draft v0.1